

# Introducción a la Ingeniería de Software

## Software

Es un set de programas y la documentación que acompaña, no es únicamente código, por lo que saber programar no alcanza para hacer software.

- 1 [Apple](#) (881.980 millones de dólares)
- 2 [Microsoft](#) (803.090 millones de dólares)
- 3 [Amazon](#) (739.460 millones de dólares)
- 4 [Alphabet](#) (711.940 millones de dólares)
- 5 [Berkshire Hathaway](#) (536.530 millones de dólares)
- 6 [Johnson & Johnson](#) (396.210 millones de dólares)
- 7 [Alibaba](#) (387.610 millones de dólares)
- 8 [Facebook](#) (378.050 millones de dólares)
- 9 [JPMorgan Chase & Co](#) (368.560 millones de dólares)
- 10 [Tencent Holdings](#) (353.670 millones de dólares)

### 1 – La destrucción del Mariner I (1962). 18,5 millones de dólares.

El del Mariner I, una sonda espacial que se dirigía a Venus, se desvió de la trayectoria de vuelo prevista poco después del lanzamiento. Desde control se destruyó la sonda a los 293 segundos del despegue. La causa fue una fórmula manuscrita que se programó incorrectamente.

### 2 – La catástrofe del Hartford Coliseum (1978). 70 millones de dólares.

Apenas unas horas después de que miles de aficionados abandonaron el Hartford Coliseum, el techo se derrumbó por el peso de la nieve. La causa: cálculo incorrecto introducido en el software CAD utilizado para diseñar el coliseo.

### 3 – El gusano de Morris (1988). 100 millones de dólares.

El estudiante de posgrado Robert Tappan Morris fue condenado por el primer ataque con "gusanos" a gran escala en Internet. Los costos de limpiar el desastre se cifran en 100 millones de dólares. Morris, es hoy profesor en MIT.

### 4 – Error de cálculo de Intel (1994). 475 millones de dólares.

Un profesor de matemáticas descubrió y difundió que había un fallo en el procesador Pentium de Intel. La sustitución de chips costó a Intel 475 millones.

### 5 – Explosión del cohete Ariane (1996). 500 millones de dólares.

En el 1996, el cohete Ariane 5 de la Agencia Espacial Europea estalló. El Ariane explotó porque un número real de 64 bits (coma flotante) relacionado con la velocidad se convirtió en un entero de 16 bits. [Te dejo un post que explica lo sucedido.](#)

### 6 – Mars Climate Orbiter (1999). 655 millones de dólares.

En 1999 los ingenieros de la NASA perdieron el contacto con la Mars Climate Orbiter en su intento que orbitase en Marte. La causa, un programa calculaba la distancia en unidades inglesas (galladas, pies y libras), mientras que otro utilizó unidades métricas.

### 7 – El error en los frenos de los Toyota (2010). 3 billones de dólares.

Toyota retiró más de 400.000 de sus vehículos híbridos en 2010, por un problema software, que provocaba un retraso en el sistema anti-bloqueo de frenos. Se estima entre sustituciones y demandas el error le costó a Toyota 3 billones de dólares.

### 8 – Las migraciones por el año 2000. 296,7 billones de dólares.

Se esperaba que el bug Y2K paralizase al mundo a la medianoche del 1 de enero 2000, ya que mucho software no había sido previsto para trabajar con el año 2000. El mundo no se acabó, pero se estima que se gastaron 206,7 billones de dólares para mitigar los daños.

## Tipos de software

- **System Software** → se encarga de controlar y gestionar los recursos del hardware de una computadora
- **Utilitarios** → subconjunto de system software que proporciona utilidades o herramientas adicionales para mejorar la función del SO.
  - *Objetivo:* Mejorar y optimizar el rendimiento del SO y de hardware de la pc.
- **Software de aplicación** → usado para realizar tareas específicas en una computadora, más enfocada en satisfacer las necesidades de los usuarios.

## Razones para no comparar software y manufactura

- Es menos predecible
- No hay producción en masa, casi ninguno es igual a otro
- No todas las fallas son errores
- No se gasta
- No va de la mano con las leyes de la física

## Problemas al desarrollar software

- La versión final no satisface las necesidades del cliente
- Difícil de extender/adaptar
- Mala documentación
- Mala calidad
- Más tiempo y costo del presupuestado → más común

## Proceso de software

Conjunto estructurado de actividades que, a raíz de un conjunto de entradas producen una salida.

Tiene como objetivo obtener un producto o servicio de software a partir de distintas entradas.

Definiciones:

- **IEEE:** Es una secuencia de pasos para un propósito dado.
- **CMM:** Conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados

## Proceso Definido

Basado en modelo industrial, un conjunto de entradas usadas en un conjunto de actividades van a producir el mismo resultado o salida siempre que las entradas sean las mismas.

Características:

- Difícil de aplicar al software, ya que no es predecible.
- No podemos decir que el software, con ciertas entradas usadas en ciertas aplicaciones van a producir las mismas salidas.
- Repetir el proceso indefinidamente y obtener los mismos resultados

## Proceso Empírico

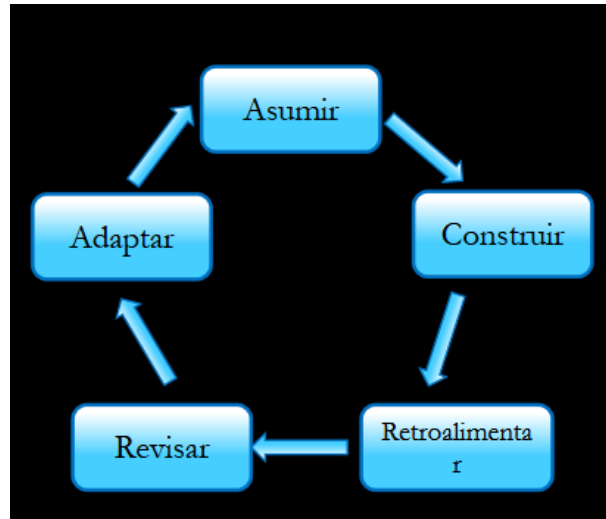
Viene del empirismo y nos centramos en la experiencia (del usuario, del negocio, de la técnica) y en el aprendizaje.

Características:

- Las variables no pueden ser controladas en su totalidad, hay que adaptarse a ellas para garantizar la calidad del software
- Busco generar cambios rápidos, e implementarlos sobre periodos, para que si no sirve no duela tirarlo.
- La experiencia solo cuenta en ese momento, en ese proyecto, para ese cliente y para el mismo equipo.

## Patrón de conocimiento

- **Hipotesis (ASUMIR)**
- **Construcción (CONSTRUIR):**
- **Retrospectiva (RETROALIMENTAR):** Inspeccionar y analizar para hacer la retrospectiva
- **Revisamos (REVISAR):** Comparar el resultado de la construcción con la hipótesis inicial.
- **ADAPTAR**



Esto es UNA repetición, o UN CICLO DE VIDA específico

## Ciclo de vida

Define cuales son las etapas y el orden de cada una. Determina cuánto de cada una de esas actividades vas a realizar a la vez, nos dice cómo usar el proceso en contexto del proyecto. Son la serie de pasos a través de los cuales un proceso o producto progresa.

## En un proyecto

Es una representación de un proceso. Gráfica una descripción del proceso desde cierta perspectiva.

## De un producto

El ciclo de vida de un proyecto termina cuando genera un producto (el ciclo de este no termina, prospera mientras exista, ya que necesitará mantenimiento)

## Tipos de ciclos de vida:

- **Secuencial:** ejecutan una etapa después de otra sin retorno.
  - Ejemplo: Primero se definen los requisitos, luego se hace el diseño, después el desarrollo, las pruebas y finalmente la entrega
- **Iterativo Incremental:** Usados por los procesos empíricos para poder adaptarse y mejorar el proceso, en cada una de las iteraciones capitalizamos lo aprendido, mejoramos la experiencia del equipo y realizamos otra iteración para incrementar el producto.
  - Hago iteraciones para ganar información y convertir esa información en un incremento que será incorporado en el sistema.
  - Ejemplo: en la primera iteración se crea el registro de gastos, en la segunda se agregan filtros y se mejora el registro, en la tercera se agregan gráficos y se mejoran los filtros.
- **Recursivo:** Usado en casos particulares, como en proyectos de alto riesgo. Toman una característica específica y en una iteración me centro en esa funcionalidad, en la otra la siguiente, etc (NO SON USADAS)
  - En espiral: Investigación de riesgos
  - En B: permite hacer una vuelta particular de cada una de las etapas e ir avanzando en los incrementos
  - Ejemplo: Se analiza un requisito, se detecta que necesita mayor detalle, se vuelve a analizar y luego se continúa.

En base a las características del proyecto se va a elegir el ciclo de vida que se va a implementar para poder llevarlo a cabo de la mejor forma posible. Siempre dependerá de qué beneficios tiene el mismo según costos y el entorno.

El **proceso** es una implementación del **ciclo de vida** que tiene un objetivo que lo guía, que es la creación de un **producto**.

## Ventajas y desventajas

### Secuencial

#### Ventajas

- Simple de organizar y entender
- Cada etapa tiene un objetivo claro
- Facilita la planificación y el seguimiento
- Permite saber en qué etapa se encuentra el proyecto

#### Desventajas

- Poco flexible ante cambios
- Si aparece un error en una etapa anterior, es difícil volver
- El usuario obtiene el producto recién al final
- No aprovecha el aprendizaje durante el desarrollo

### Iterativo Incremental

#### Ventajas

- Permite adaptarse a cambios
- Permite aprender durante el desarrollo

- Cada iteración aporta información nueva
- El producto crece progresivamente mediante incrementos
- Permite incorporar el feedback y mejorar lo analizado

#### Desventajas

- Requiere adaptación constante
- Puede generar retrabajo
- Es más difícil saber exactamente cómo será el producto final desde el comienzo
- Requiere una buena organización entre las iteraciones

#### Recursivo

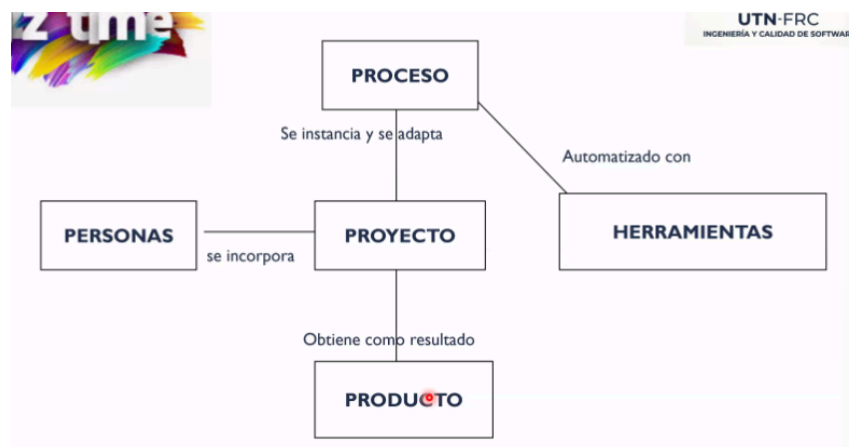
#### Ventajas

- Permite concentrarse profundamente en características específicas
- Permite tratar aspectos complejos o de alto riesgo
- Facilita analizar cada característica de manera detallada

#### Desventajas

- Puede ser más complejo de gestionar
- Puede requerir mucho tiempo
- Puede generar muchas vueltas sobre una misma característica
- No es un enfoque usado habitualmente

## Proyecto de Sistemas de Información



Un **proyecto** es llevado a cabo por **personas** que implementan **herramientas** para automatizar los procesos y que tiene como resultado un **producto**.

**Proceso:** definición abstracta que se materializa a través de los **proyectos**, se adapta a las necesidades concretas del mismo. En él se expresa en forma teórica lo que se debe hacer para hacer software y debo adaptarlo al proyecto (si es un proceso definido) o terminar de armarlo (si es un proceso empírico).

Los procesos toman como entrada requerimientos, y a través de un conjunto de actividades (guiadas y estructuradas por un objetivo) obtienen como salida un producto.

**Proyecto:** unidad organizativa de gestión, necesita de personas, recursos y procesos para poder existir.

Características

- **Duración limitada:** son temporales, cuando se alcanza el objetivo, el proyecto termina. (Tiene un inicio y un fin)
- **Orientado a un objetivo:** tengo que poder definir hacia dónde voy y poder medir cuando se alcance el objetivo para poder finalizarlo. Tienen un único objetivo que tiene que ser claro, no ambiguo y alcanzable. (Esto me permite que los proyectos sean únicos)
- **Son únicos:** todos los proyectos similares que tienen características que los hacen únicos.
- **Tiene tareas relacionadas entre sí basadas en esfuerzos y recursos para alcanzar el objetivo:** las tareas dependen una de otra, tienen precedencia y asociados recursos y esfuerzos

Roles de un proyecto



- **Líder o Project Manager (PM):** se ocupa de todas las tareas de gestión que hacen que el equipo del proyecto sea guiado a lograr el objetivo.  
Es el encargado de administrar todos los recursos y hacer el seguimiento de la planificación verificando que todo salga bien.
- **Equipo del proyecto:** Grupo de personas comprometidas en alcanzar un conjunto de objetivos los cuales son responsables, se complementan (intercambian conocimientos y habilidades) para trabajar en equipo y lograrlo.

Planificación

**1. Definición del alcance:**

| ALCANCE DEL PROYECTO                                                                                                           | ALCANCE DEL PRODUCTO                                                     |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Todo trabajo que debe hacerse para entregar el producto con todas las características y funciones especificadas en el objetivo | Todas las características que pueden incluirse en un producto o servicio |

**2. Definición del proceso y ciclo de vida:**

El proceso de desarrollo es el conjunto de actividades que necesito ejecutar para construir el producto de software y el ciclo de vida es de qué manera ejecutar esas actividades.

**3. Estimación:**

Estimar todas las características necesarias para la planificación con cierto nivel de incertidumbre, se definen las siguientes características.

- Tamaño
- Esfuerzo
- Calendario
- Costo
- Recursos críticos

**4. Gestion de riesgos:**

Identificar todos los riesgos probables de ocurrir o los más impactantes. Calcular la exposición de riesgo (multiplicando las variables de impacto y ocurrencia), con esto determinamos los riesgos principales y cuales voy a gestionar para disminuir el impacto.

**5. Asignación de recursos**

**6. Programación de proyectos**

**7. Definición de métricas:**

|                    |                                                                                                                                                                                                                                                                |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>DE PROCESO</b>  | Mismas métricas del proyecto pero despersonalizadas de uno en particular.<br>Nos deja saber si como organización estamos trabajando bien o mal.<br>Ej: Porcentaje de proyectos que terminan en término y porcentaje de proyectos que terminan fuera de término |
| <b>DE PROYECTO</b> | Nos permiten saber si un proyecto de software en ejecución se está cumpliendo de acuerdo a lo planificado o no<br>Ej: Comparación del tiempo calendario real con el planificado                                                                                |
| <b>DE PRODUCTO</b> | Miden características que tienen una relación directa con el producto que estamos construyendo<br>Ej: Métricas de defectos, todas las métricas de tamaño                                                                                                       |

**8. Monitoreo y control:**

El PM determina si se cumple o no lo establecido en el plan de proyecto.

## Gestión de productos

Se centra en la definición, planificación y evolución de un artefacto informático construido para satisfacer una necesidad específica basándose en requerimientos concretos.

Tiene como objetivo priorizar la creación de valor para el cliente y evitar la inversión de esfuerzo en elementos considerados como desperdicio.

## Pilares de la gestión de un producto de software

- **Enfoque en el valor real:** Casi el 80% de las características de un software no se usan, y solo el 7% siempre son usadas. El desafío de la gestión de producto es encontrar y enfocarse únicamente en ese 7% que aporta valor.

- **Evolución del producto:** Se inicia con una base **funcional**. Luego, el gestor, debe guiar la evolución para que sea **confiable, usable, conveniente** y placentero y **significativo** para el usuario.
- **Propósito claro:** Los productos se gestionan y desarrollan para alcanzar objetivos específicos.
- **Planificación ágil:** La visión integral del producto se centraliza y materializa en el *Product Backlog*. Esta luego se descompone en entregas frecuentes (*releases*) y se van construyendo mediante iteraciones sucesivas.
- **Productividad redefinida:** Asegurarse que se está construyendo lo correcto todos los días.
- **Uso de herramientas de validación temprana:** Definir hipótesis y validarlas a través de un MVP (Producto Mínimo Viable) para recopilar aprendizaje sobre los clientes con el menor esfuerzo. También se emplean metodologías como Design Thinking (empatía y necesidades del usuario) y Lean UX (ciclo construir-medir-aprender).

## MVP - Mínimo Producto Viable

Concepto de Lean Startup que se enfoca en el impacto del aprendizaje en el desarrollo de nuevos productos.

Es una versión de un nuevo producto que permite a un equipo recopilar la cantidad máxima de aprendizaje validado sobre los clientes con el menor esfuerzo posible.

- Se basa en la idea de que producir un producto real para observar el comportamiento real de la gente es mucho más confiable que preguntarles qué harían.
- El producto, para que sea un MVP, debe tener el valor suficiente para que las personas estén dispuestas a usarlo o comprarlo, debe tener beneficios futuros para los usuarios y proporcionar un ciclo de retroalimentación que guíe el desarrollo futuro.
- Permite probar 2 hipótesis:
  - Hipótesis de valor: Si el producto realmente entrega valor al cliente.
  - Hipótesis de crecimiento: Como descubrirán el producto nuevos clientes

### **MVP**

- Versión de un **nuevo producto** creado con el **menor esfuerzo posible**
- Dirigido a un **subconjunto de clientes potenciales**
- Utilizado para obtener **aprendizaje validado**.
- Más cercano a los **prototipos que a una versión real funcionando de un producto**.

### **MMF**

- es la **pieza más pequeña de funcionalidad** que puede ser liberada
- tiene valor tanto para la organización como para los usuarios.
- Es parte de un MMR or MMP.

### **MMP**

- Primer release de un MMR dirigido a **primeros usuarios** (early adopters),
- Focalizado en características clave que satisfarán a este grupo clave.

### **MMR**

- Release de un producto que tiene el conjunto de características más pequeño posible.
- El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.
- **MMP = MMR1**

## Product Backlog

Artefacto fundamental en Scrum y Agile

Es una lista emergente, ordenada y priorizada de características y requerimientos (ítems o PBIs) que se necesitan para mejorar el producto.

El *PO* es el encargado de definir y priorizar la lista, definiendo qué requerimientos se necesitan primero según el valor de negocio que aportan.

El Product Backlog nunca está definido al 100%, ya que a medida que el proyecto transcurre, los ítems pueden borrarse, agregarse o ser re-priorizados.

## Design Thinking

Metodología iterativa para crear servicios centrados en el usuario, tiene cinco etapas obligatorias:

- Empatizar
- Definir
- Idear
- Prototipar
- Probar

## Lean UX

Enfoque sustentado en el circuito **Construir - Medir - Aprender**, tiene como objetivo permitir equivocarse en estadios precoces cuando es barato reconducir la idea

# Requerimientos Ágiles

Está centrada en construir valor de negocio y satisfacer al cliente mediante la entrega temprana y continua de software útil. El agilismo asume que la incertidumbre es inevitable y que los requerimientos irán madurando y descubriendo a medida de que el proyecto avanza.

## Fundamentos

- **Enfoque en el valor:** Construir el producto que resuelva un problema y aporte valor al negocio del cliente, no únicamente cumplir un plan o entregar documentos.
- **Solo lo suficiente (Just in Time):** Los requerimientos se detallan y analizan únicamente cuando se necesitan. Se empieza con una visión general y se profundiza iterativamente para generar una retroalimentación rápida.
- **Evitar el BRUF (Big Requirements Up Front):** Las metodologías ágiles rechazan la práctica de especificar el 100% de los requerimientos al inicio del proyecto, porque esto genera desperdicio frente a la alta probabilidad de que las necesidades del usuario cambien.

## Triple Restriccion Agil

- **Alcance Variable:** El tiempo y los recursos son fijos, mientras que el alcance es variable.
- **Compromiso de Entrega:** Se evalúa, en función del tiempo y de los recursos disponibles, qué porción de funcionalidades de mayor valor puede entregarle al cliente en cada ciclo.

## Otros fundamentos (Comunicacion, tipos, gestión del cambio)

- **Cara a Cara sobre la documentación:** El medio de comunicación más eficiente es la conversación directa entre técnicos y no técnicos (que trabajarán en conjunto durante todo el proyecto). La documentación formal (ERS) se reemplaza por la disponibilidad del cliente y las User Stories.
- **Dominio del Problema:** Nos interesa trabajar los *requerimientos de Negocio* y los *requerimientos de Usuario*, dejando los requerimientos funcionales, no funcionales y de implementación como tareas de transformación para el equipo.
- **Abiertos al Cambio:** *“El usuario dice lo que quiere cuando recibe lo que pidió”*, por eso se aceptan cambios en los requerimientos, incluso en etapas finales del desarrollo, usando este factor como una ventaja competitiva.\

## User Story

Tecnica para trabajar requerimientos de usuario en entornos agiles.

Es una descripción corta de una funcionalidad esperada del producto que aporta valor a un usuario del sistema.

Funciona como recordatorio para el equipo de desarrollo y el PO de que deben conversar en el futuro sobre una necesidad de negocio.

## Enfoque

- **Porciones verticales:** Las US se consideran porciones verticales del producto, porque para entregar una característica terminada, deben abarcar todas las capas de la arquitectura (UX, lógica de negocio y DB)
- **Multipropósito:** Representan una necesidad del usuario, son una descripción del producto, sirven como ítem de planificación y actúan como un mecanismo para diferir una conversación detallada hasta el momento adecuado.

## Las “3 C”

- **Tarjeta (Card):** Es la parte visible y escrita que contiene la descripción de la historia y se usa para planificar.

- **Conversación (Conversation):** Parte más importante y no visible, es el diálogo directo entre el PO y el equipo de desarrollo, donde se negocian y desarrollan los detalles específicos para implementar la US.
- **Confirmación (Confirmation):** Son las pruebas de aceptación y condiciones que determinan cuando la historia está completa y cumple lo esperado por el PO.

## Formato

Se escriben como:

*“Como <ROL> yo puedo <ACTIVIDAD> de tal forma que <VALOR DE NEGOCIO QUE RECIBO>”*

El **ROL** representa a quien hará la acción o recibirá el beneficio.

La **ACTIVIDAD** es la acción del sistema.

El **VALOR DEL NEGOCIO** es la justificación de por que esa acción es necesaria y permite priorizarla.

## Criterios y pruebas de aceptación

- **Criterios de aceptación:** Definen los límites de la historia y ayuda a tener una visión de los requerimientos funcionales mínimos. Son redactados a altos niveles, indicando una intención.
- **Pruebas de aceptación:** Expresan detalles resultantes de las conversaciones. Se prueban escenarios exitosos y fallidos, y el PO acepta la US solo si se cumplen todas esas pruebas.

## Filtros de calidad

- **Definition of Ready:** Medida de calidad que determina si la US está lo suficientemente bien formulada como para ingresar a una iteración de desarrollo.
- **Modelo INVEST:** Estándar de calidad mínimo para que la US cumpla con el DoR. Dice que la historia debe ser: Independiente, Negociable, Valuable, Estimable, Small y Testeable.
- **Definition of Done:** Check List que establece cuando la US fue implementada correcta y completamente para ser mostrada al PO y pasar a producción.

## Niveles de Abstracción y Casos Especiales

- **Epica:** US de granularidad gruesa, demasiado grande para entrar en una iteración, debe ser dividida en US mas pequeñas para implementarse.
- **Tema:** Conjunto de US agrupadas porque están relacionadas, se tratan como una entidad simple a la hora de estimar o planificar.

- **Spike:** US que se genera cuando existe una incertidumbre técnica o funcional que impide estimar su tamaño. Son una excepción a la regla, necesitan ser investigadas o prototipadas, y se usan para reducir riesgos antes de desarrollarlas.

## Estimaciones Agiles

Aproximaciones usadas para predecir la completitud de un trabajo y gestionar los riesgos del proyecto, asumiendo de que la **incertidumbre es inevitable**.

Las estimaciones no son planes, promesas ni compromisos formales en AGILE.

## Diferencias con el Enfoque Tradicional

- **Medidas Relativas vs Absolutas:** En lugar de intentar predecir con precisión absoluta (costoso y propenso a errores), el agilismo estima mediante la **comparación** relativa de tamaños, ya que las personas no son buenas estimando en términos absolutos, pero si comprando magnitudes.
- **Quien Estima:** En el enfoque tradicional estima un gerente o líder, en el ágil, *estima quien hace el trabajo*. El equipo de desarrollo realiza la estimación de forma colectiva y colaborativa. El PO participa aclarando dudas, pero no estima.
- **Ajuste por Alcance:** Como en las metodologías ágiles el tiempo y los recursos son fijos, mientras el alcance es variable, una falla en la estimacion no implica extender el tiempo del proyecto, sino que se ajusta el alcance entregado

## Unidad de medida → Story Points

- Es la unidad de medida que se usa para medir el tamaño relativo de una US frente a otra.
- No mide tiempo, sino que combina tres variables:
  - *Incertidumbre:* cuanta informacion falta sobre el negocio.
  - *Complejidad:* cantidad de partes y relaciones que tiene la funcionalidad.
  - *Esfuerzo:* horas de trabajos ideales o lineales requeridas.
- El tamaño depende de la duración o el esfuerzo absoluto, ya que el esfuerzo real puede variar dependiendo de las habilidades y la experiencia de la persona que realice la tarea.

## Metrica de progreso: Velocity

- Metrica principal de progreso del equipo, no se estima, sino que se **calcula o mide**.
- Calculo: Suma de SP asignados a las US que el equipo completo al 100% y fueron aceptadas al PO al finalizar un Sprint.
- Unicamente se cuentan los SP finalizados, o los que no lo estan o estan parcialmente.

- Nos permite predecir con cuanto trabajo se pueden comprometer en futuras iteraciones y ayuda a derivar la duracion total del proyecto.

## Poker Planning | Tecnica de estimacion

- Tecnica colaborativa basada en cartas que combina el juicio experto, la analogia y la tecnica Delphi, promoviendo que el equipo debata y llegue a un consenso sobre el tamano de cada historia.
- **Historia canonica:** Se define primero una historia de referencia simple (baja incertidumbre) y se le asigna un puntaje base (1, 2 SP) para comparar todas las demas, con ella.
- **Escala de Fibonacci:** Para los puntajes se usa la serie de Fibonacci (1, 2, 3, 5, 8, 13, etc.), porque la complejidad e incertidumbre del software suele incrementarse en forma exponencial.
- **Limites:** Si a una historia se le asigna un puntaje muy alto (13 o mas), significa que estamos ante una US **Epica** para entrar en un solo sprint, y debera ser dividida en partes mas pequenas.

## Reestimacion

Solo se aconseja reestimar una US si la comprension del equipo cambio al obtener mas informacion.

No se debe reestimar si el equipo avanza mas lento de lo esperado ya que las inexactitudes en los tiempos se equilibran solas con la metrica de velocidad.

## Framework Scrum

**Scrum:** framework ligero, disenado para ayudar a personas, equipos y organizaciones a generar valor a traves de soluciones adaptables para problemas complejos.

- No proporciona instrucciones detalladas, proporciona reglas que guian las interacciones basandose en el **empirismo** y el **pensamiento Lean**.

## Pilares de SCRIM

- **Transparencia:** El proceso y el trabajo emergente debe ser completamente visible para los que realizan el trabajo y los que lo reciben, evitando decisiones basadas en informacion oculta.
- **Inspeccion:** Los artefactos y el progreso hacia los objetivos deben revisarse frecuentemente para detectar variaciones indeseadas.
- **Adaptacion:** Si se detectan desviaciones fuera de los limites aceptables, el proceso debe reajustarse lo antes posible.

- **Valores de Scrum:** Para que estos pilares funcionen, el equipo debe interiorizar cinco valores:
  - Compromiso
  - Enfoque
  - Apertura
  - Respeto
  - Coraje

## Roles del SCRUM

Un equipo de scrum es una unidad cohesionada, multifuncional y autogestionada. No hay subequipos ni jerarquias, son de 10 o menos personas.

- **Product Owner:** Responsable de maximizar el valor del producto y gestionar el Product Backlog. Representa las necesidades de los stakeholders o del negocio.
- **Scrum Master:** Es un lider del equipo y organizacion, asegura que el Scrum se entienda y se aplique bien, promueve la autogestion, facilita eventos y elimina impedimentos que bloqueen al equipo.
- **Desarrolladores:** Encargados de crear el incremento util y de valor de cada Sprint, son los responsables de crear el Sprint Backlog, asegurar la calidad y adaptar el plan diariamente.

## Eventos y Timebox

Todo evento Scrum esta limitado por un Timebox, para obligar al equipo a no perder tiempo y a negociar el alcance en vez de extender las fechas.

- **Sprint:** Evento contenedor de todas las ceremonias, funcionando como un proyecto corto con duracion fija (1 mes o menos).
- **Sprint Planning:** Inicia el Sprint definiendo el trabajo a realizar, responde a tres preguntas: (Max. 8hs para un sprint de 1 mes)
  - Por que es valioso este Sprint? → Objetivo
  - Que se puede hacer? → Items seleccionados
  - Como se realizara? → Plan de accion
- **Daily Scrum:** Reunion corta (15m), misma hora y lugar, exclusiva para desarrolladores. Sirve para inspeccionar el objetivo y adaptar el trabajo para el dia siguiente.
- **Sprint Review:** Sesion de trabajo donde el equipo presenta los resultados del Sprint a los stakeholders. Sirve para inspeccionar lo logrado, recibir feedback y adaptar el Product Backlog para el futuro (Max. 4hs para un sprint de 1 mes)

- **Sprint Retrospective:** Cierre del Sprint. Reunion interna del equipo para ver como trabajaron en terminos de individuos, procesos y herramientas, con el fin de mejorar para el proximo ciclo. (Max. 3hs para un sprint de 1 mes)

## Artefactos

Representan trabajo o valor y estan disenados para maximizar la transparencia.

- **Product Backlog:** Lista emergente y priorizada de todo lo necesario para mejorar el producto:
  - *Compromiso → Objetivo del producto: Describe el estado futuro del producto a largo plazo que se quiere alcanzar.*
- **Sprint Backlog:** Plan en tiempo real creado por y para los desarrolladores, compuestos por los items seleccionados para la iteracion actual y el plan de accion.
  - *Compromiso → Objetivo del Sprint*
- **Incremento:** Un paso concreto y utilizable que se suma a los incrementos anteriores para acercarse al Objetivo del Producto
  - *Compromiso → Definition of Done (DoD)*

## SCM | Gestion de Configuracion de Software

Disciplina de soporte transversal a todo el proyecto.

Su objetivo es establecer y mantener la integridad del producto a lo largo de todo su ciclo de vida.

Surge como medida preventiva para resolver graves problemas de desarrollo (perdida de componentes, de cambios, superposicion de modificaciones, fallas de regresión, doble mantenimiento).

## Conceptos

- **Item de Configuracion (IC):** Todo artefacto que forma parte del producto o proyecto, susceptibles a sufrir cambios, y que deben ser almacenados en un repositorio para conocer su estado.
  - Ejemplos:Codigo fuente, arquitectura, casos de prueba, manuales de usuario.
  - Se clasifican en *items de producto, de proyecto o de iteracion.*
- **Version:** Instancia particular de un IC que difiere de otras en un momento especifico del tiempo.
- **Variante:** Version de un IC que evoluciona de manera separada para representar configuraciones alternativas (Ej. adaptar el mismo software para otro SO).
- **Configuracion:** Conjunto de todos los IC con su version especifica, en un momento dado, funciona como una foto del sistema.

- **Repositorio:** Contenedor (centralizado en un servidor o descentralizado en multiples clientes) que mantiene la historia de cada item, sus atributos y relaciones.
- **Linea Base:** Conjunto de IC que fueron contruidos, revisados y aprobados. Sirve como un punto de referencia estable para demostrar madurez y usarse como base para otros desarrollos. Existen lineas base de *especificacion* (documentos, requerimientos) y *operacionales* (codigo ejecutable que paso por controles de calidad)
- **Rama y Merge:** Las ramas permiten bifurcar el desarrollo de un software para experimentar o resolver de forma paralela. El merge ocurre cuando se fusionan esas configuraciones hacia una rama principal estable.

## Actividades

- **Identificacion de Items:** Asignar un ID a cada artefacto, definiendo convenciones y reglas de nombrado, estructura y ubicacion dentro del repositorio.
- **Control de Cambios:** Procedimiento formal para alterar una Baseline. Dirigido por el *Comite de Control de Cambios*, que recibe la propuesta, la analiza (impacto tecnico y de recursos) y la autoriza o rechaza. Tambien notifica a los interesados.
- **Auditorias de Configuracion:** Controles objetivos realizados por un auditor externo al equipo de desarrollo, valida la integridad y calidad. La *auditoria fisica* verifica que los IC del repositorio coincidan con el plan y el nombrado. La *auditoria funcional* evalua la trazabilidad.
- **Informes de Estado:** Dan un registro automatizado sobre la evolucion del sistema y las modificaciones de las baseline, asegurando que todos los involucrados trabajen en la ultima version.

## SCM en Ambientes Agiles

SCM en Agile, cambia su enfoque, busca hacer un seguimiento y coordinar el desarrollo. Al ser equipos auto-organizados, se evitan las auditorias externas y el comite formal de cambios en el Sprint, priorizando la adaptacion constante y la automatizacion intensiva.

- **Integracion Continua (CI):** Practica donde los desarrolladores integran su codigo a un repositorio. El codigo se verifica automaticamente mediante pruebas unitarias para detectar problemas rapidamente.
- **Entrega Continua (CD):** Garantiza que el software se construya de tal manea que siempre este liberado a produccion, incluye automatizacion de pruebas de aceptacion, requiere intervencion humana para realizar el pase final.

- **Despliegue Continuo:** El software modificado y aprobado pasa al entrono de produccion sin ninguna intervencion humana, usando *pipelines* (flujos de trabajo) automatizados.
- **Estrategias de Despliegue:** Se usan tecnicas como *Canary Deployment* (libera la actualizacion a una porcion de usuarios para probar su estabilidad antes de extenderla a todos) o *Blue-Green Deployment* (Se transfiere el trafico gradualmente a un entorno azul, con la version vieja, a un entorno verde identico con la version nueva)